



EC-220 Computer System Architecture

Dr. Ahsan Shahzad

Department of Computer and Software Engineering (DC&SE)

College of Electrical and Mechanical Engineering (CEME)

National University of Science and Technology (NUST)

Week # 3 & 4

History of MIPS

- **Microprocessor without Interlocked Pipeline Stages (MIPS)**
- MIPS developed at Stanford by **Hennessy et al.**
- MIPS Computer systems founded in 1984, Now **MIPS Technologies**
- Multiple versions of MIPS: including **MIPS I, II, III, IV, and V**; as well as five releases of **MIPS32/64**
- MIPS32 is based on MIPS II with some additional features from MIPS III, MIPS IV, and MIPS V
- MIPS64 is based on MIPS V

What is interlock in pipeline?

The **hardware to detect a data hazard and stall the pipeline** until the hazard is cleared is called a pipeline interlock

MIPS ISA

- **Microprocessor without Interlocked Pipeline Stages (MIPS)**
- Thirty-two **32-bit** general-purpose registers (GPR).
- **Register \$0** is hardwired to zero and writes to it are discarded.
- **Register \$31** is the link register.
- The **program counter (PC)** specifies the address of the next Instruction.
- The **exception program counter (EPC)** register remembers the program counter when there is an interrupt or exception.
- For integer multiplication and division instructions, which run asynchronously from other instructions, a pair of 32-bit registers, **HI and LO** are provided.

MIPS Assembly Language

- Lets Start,

MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add \$s1,\$s2,\$s3	$\$s1 = \$s2 + \$s3$	Three register operands
	subtract	sub \$s1,\$s2,\$s3	$\$s1 = \$s2 - \$s3$	Three register operands
	add immediate	addi \$s1,\$s2,20	$\$s1 = \$s2 + 20$	Used to add constants
Data transfer	load word	lw \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Word from memory to register
	store word	sw \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Word from register to memory
	load half	lh \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Halfword memory to register
	load half unsigned	lhu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Halfword memory to register
	store half	sh \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Halfword register to memory
	load byte	lb \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Byte from memory to register
	load byte unsigned	lbu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Byte from memory to register
	store byte	sb \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Byte from register to memory

MIPS Operands

- limited number of special locations built directly in hardware called *registers*.
- Visible to the programmer
- **Variables** in high-level language usually **maps to Registers** in low level languages.
- **Complex Data structures** i.e. Arrays, Structures are kept/map in memory
- Memory is just a large, single dimension array, which is byte addressable

MIPS operands

Name	Example	Comments
32 registers	<code>\$s0-\$s7, \$t0-\$t9, \$zero, \$a0-\$a3, \$v0-\$v1, \$gp, \$fp, \$sp, \$ra, \$at</code>	Fast locations for data. In MIPS, data must be in registers to perform arithmetic, register <code>\$zero</code> always equals 0, and register <code>\$at</code> is reserved by the assembler to handle large constants.
2^{30} memory words	<code>Memory[0], Memory[4], . . . , Memory[4294967292]</code>	Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers.

MIPS Assembly Language

- Compiling the C Statement using Registers $f = (g+h) - (i+j)$

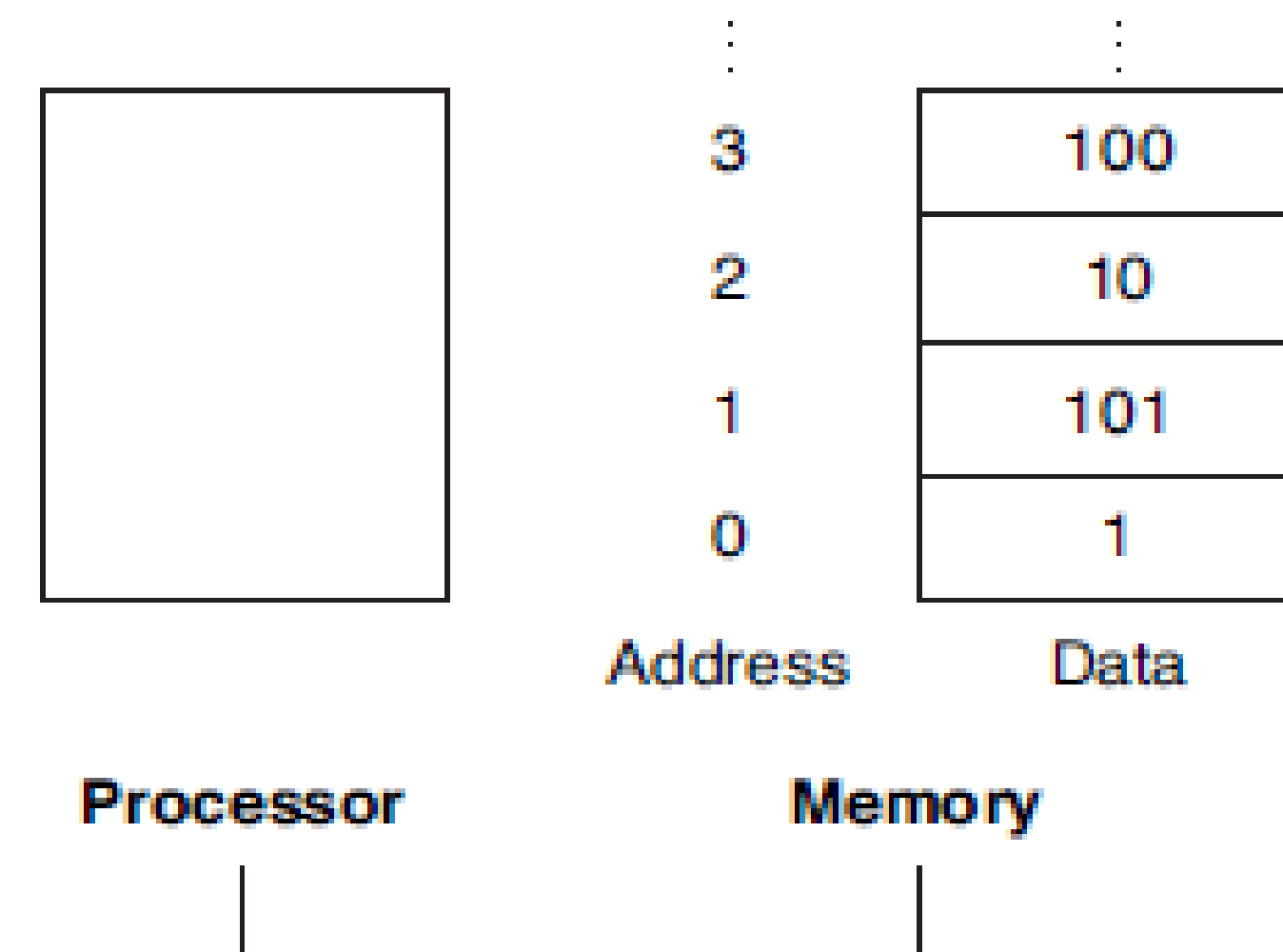
The variables **f**, **g**, **h**, **i**, and **j** are assigned to the registers **\$S0**, **\$S1**, **\$S2**, **\$S3**, and **\$S4**, respectively. What is the compiled MIPS code?

ANSWER:

```
add $t0,$s1,$s2 # register $t0 contains g + h
add $t1,$s3,$s4 # register $t1 contains i + j
sub $s0,$t0,$t1 # f gets $t0 - $t1, which is (g + h)-(i + j)
```

Memory Operands

- **Arithmetic Instructions** can work with **register operands only**
- Complex data structures (**Arrays, Structures**) are kept in memory
- **Data transfer instructions**
 - Transfer of data between memory & registers
 - **Load:** bring data into reg. from memory
 - **Store:** store data in memory



MIPS Assembly Language

Compiling an Assignment when an Operand is in Memory $g = h + A[8]$

Let's assume, A in an array of 100 words and that the compiler has associated the variables **g** and **h** with the registers **\$S1** and **\$S2**, respectively. Moreover, starting or base address of array is in **\$S3**. What is the compiled MIPS code?

ANSWER:

```
lw  $t0, 8($S3)  # Temporary reg. $t0 gets A[8]
add $S1, $S2, $t0  # g = h + A[8]
```

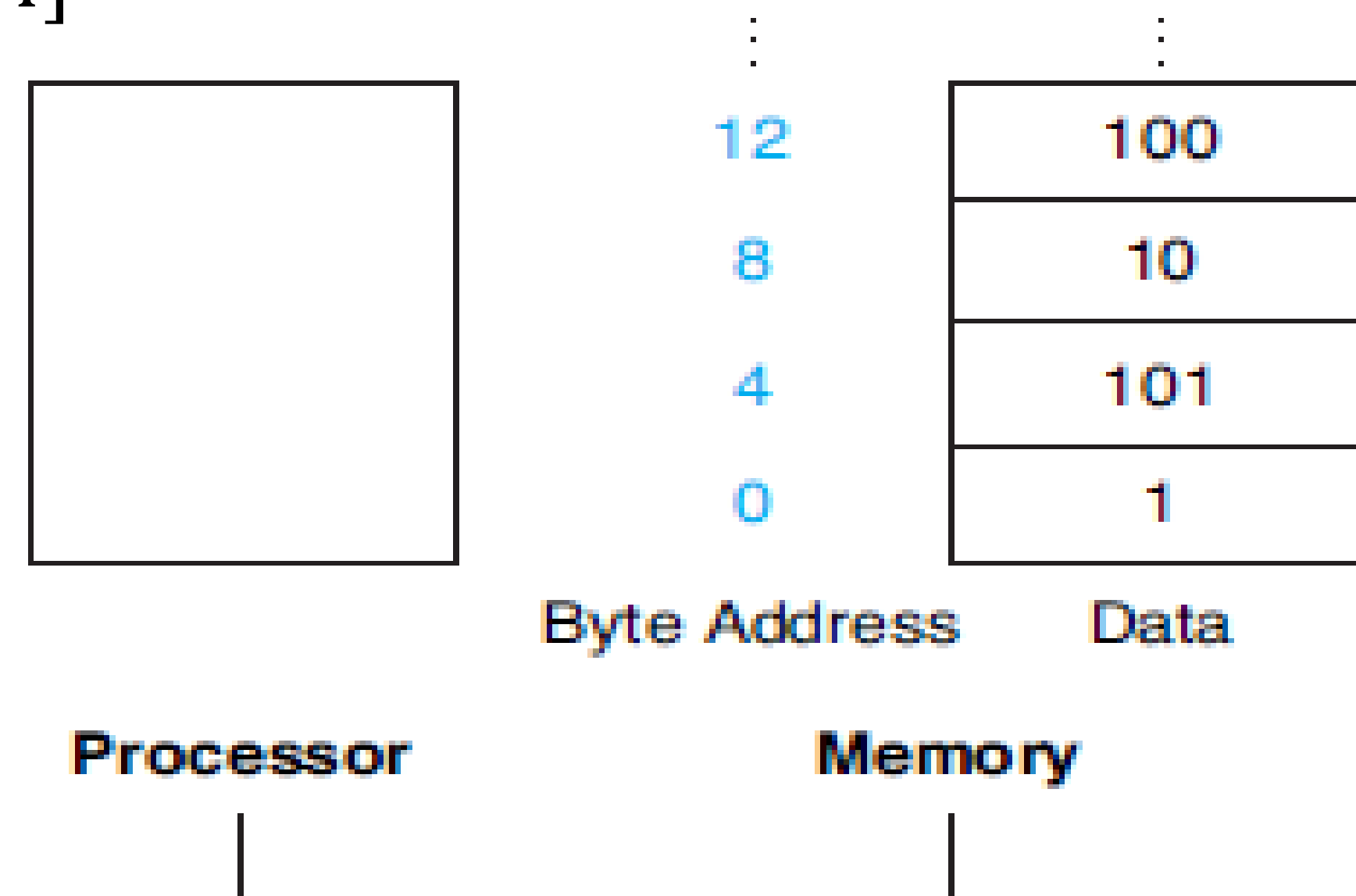
Is it right?

Actual Addressing

- Architectures address individual bytes.
- Byte addressing also affects the array index.
- To get the proper byte address in the code above, the offset to be added to the base register \$s3 must be 4 x 8, or 32.
- So, that the load address will select A[8] and not A[8/4]

ANSWER:

```
lw  $t0, 32($S3)
add $S1, $S2, $t0  # g = h + A[8]
```



Constant or Immediate Operands

- Constants occur frequently
- `lw $t0, AddrConstant4($s1)` `# $t0 = constant 4`
- `add $s3, $s3, $t0` `# $s3 = $s3 + $t0 ($t0 == 4)`

OR

- **`addi $s3, $s3, 4`** **`# $s3 = $s3 + 4`**

Number Systems

Numbering System

System	Base	Digits
Binary	2	0, 1
Octal	8	0,1,2,3,4,5,6,7
Decimal	10	0,1,2,3,4,5,6,7,8,9
Hexadecimal	16	0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

Decimal Base-10	Binary Base-2	Octal Base-8	Hexa Decimal Base-16
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Conversion b/w Number Systems

Decimal to Binary or Hexadecimal System

$$25d = 2 * 12 + 1$$

$$12d = 2 * 6 + 0$$

$$6d = 2 * 3 + 0$$

$$3d = 2 * 1 + 1$$

$$1d = 2 * 0 + 1$$

$$25d = 11001b$$

$$35d = 16 * 2 + 3$$

$$2d = 16 * 0 + 2$$

$$35d = 23h$$

Conversion b/w Number Systems

Binary or Hexadecimal to Decimal system

$$\begin{aligned} 101011b &= 1*2^0 + 1*2^1 + 0*2^2 + 1*2^3 + 0*2^4 + 1*2^5 \\ &= 1 + 2 + 0 + 8 + 0 + 32 \\ &= 43d \end{aligned}$$

$$\begin{aligned} 123h &= 3*16^0 + 2*16^1 + 1*16^2 \\ &= 3 + 32 + 256 \\ &= 291d \end{aligned}$$

Second Method: Nested Multiplication

$$101011b = (1 \times 2 + 0) \rightarrow 2 \times 2 + 1 \rightarrow 5 \times 2 + 0 \rightarrow 10 \times 2 + 1 \rightarrow 21 \times 2 + 1 \rightarrow 43d$$

$$123h = 1 \times 16 + 2 \rightarrow 18 \times 16 + 3 \rightarrow 291d$$

Conversion b/w Number Systems

Binary <-> Hexadecimal system

1 0 1 1 | 1 1 0 0b =>

1 1 0 0b = 12d = 0Ch

1 0 1 1b = 11d = 0Bh

10111100b = 0BCh

- Direct translation of nibbles (4 bits) to hex digits and vice versa.
- More convenient to use hex digits

Integer Representations in Computer

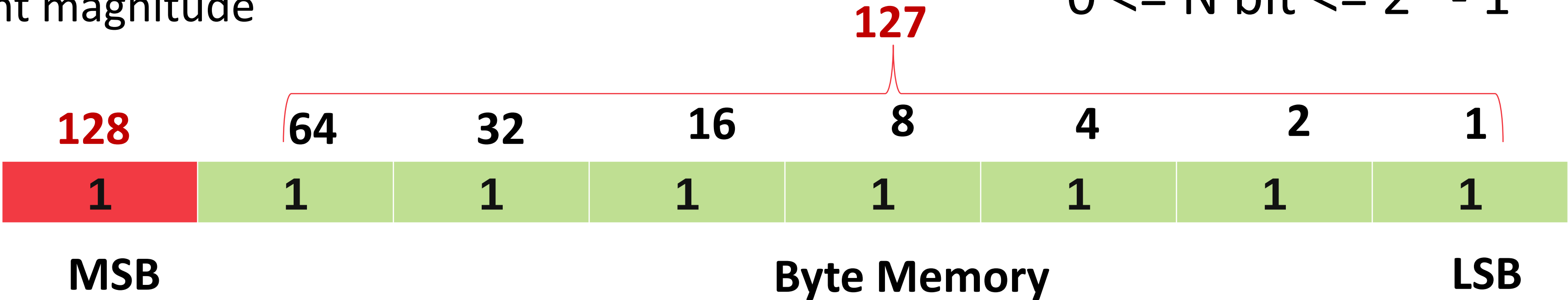
Unsigned Integer

- Positive numbers, represent magnitude

$$0 \leq 8 \text{ bit} \leq 255$$

$$0 \leq 16\text{bit} \leq 65535$$

$$0 \leq N \text{ bit} \leq 2^N - 1$$



Signed Integer

- Positive (MSB = 0) or Negative (MSB = 1)
- Negative integers are stored in computer in 2's complement form

00000110 -> 6d
11111001
+1
11111010 -> -6d

00000001 -> 1d
11111110
+1
11111111 -> -1d

$$-128 \leq 8 \text{ bit} \leq 127$$

$$-32768 \leq 16\text{bit} \leq 32767$$

$$-(2^{N-1}) \leq N \text{ bit} \leq 2^{N-1} - 1$$

Sign Extension

- In MIPS assembly language, to copy the sign repeatedly to fill the rest of the register—called ***sign Extension***
- its *purpose* is to place a correct representation of the number within that register.

MIPS Instruction Format

- Instructions are divided into three types: **R, I and J**.
- Every instruction starts with a **6-bit opcode**.

Type	-31-	format (bits)					-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)	
I	opcode (6)	rs (5)	rt (5)	immediate (16)			
J	opcode (6)	address (26)					

In addition to the opcode,

- **R-type** instructions specify **three registers, a shift amount field, and a function field**;
- **I-type** instructions specify **two registers and a 16-bit immediate value**;
- **J-type** instructions follow the opcode with a **26-bit jump target**

MIPS Assembly and Instructions Format

- **R-format**

- **OP** rd, rs, rt OR **OP** rd, rt, shamt
- EX: **add** \$S1, \$S2, \$S3 OR **sll** \$S1, \$S2, 2

- **I-format**

- **OP** rt, IMM(rs) OR **OP** rs, rt, IMM
- EX: **lw** \$S1, 4(\$S2) OR **addi** \$S1, \$S2, 6

- **J-format**

- **OP** Label
- EX: **J** Top_Label

Type	-31-	format (bits)					-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)	
I	opcode (6)	rs (5)	rt (5)	immediate (16)			
J	opcode (6)	address (26)					

Representing Instructions in Computer

- In MIPS assembly language, registers **\$s0 to \$s7** map onto **registers 16 to 23**, and **registers \$t0 to \$t7** map onto **registers 8 to 15**.

- Conversion of MIPS Assembly to Machine Code:

add \$t0,\$s1,\$s2

- The decimal representation is

0	17	18	8	0	32
----------	-----------	-----------	----------	----------	-----------

- The binary representation is

6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
000000	10001	10010	01000	00000	100000

Arithmetic and Data Transfer Instructions

- Instructions are divided into three types: R, I and J.

Instruction	Format	op	rs	rt	rd	shamt	funct	address
add	R	0	reg	reg	reg	0	32 _{ten}	n.a.
sub (subtract)	R	0	reg	reg	reg	0	34 _{ten}	n.a.
add immediate	I	8 _{ten}	reg	reg	n.a.	n.a.	n.a.	constant
lw (load word)	I	35 _{ten}	reg	reg	n.a.	n.a.	n.a.	address
sw (store word)	I	43 _{ten}	reg	reg	n.a.	n.a.	n.a.	address

Example:

If **\$t1** has the base of the **array A** and **\$s2** corresponds to **h**, the assignment statement

$$A[300] = h + A[300];$$

is compiled into

```
lw    $t0,1200($t1)    # Temporary reg $t0 gets A[300]
add   $t0,$s2,$t0      # Temporary reg $t0 gets h + A[300]
sw    $t0,1200($t1)    # Stores h + A[300] back into A[300]
```

What is the MIPS machine language code for these three instructions?

Op	rs	rt	rd	Add ress	funct
35	9	8	1200		
0	18	8	8	0	32
43	9	8	1200		

Op	rs	rt	rd	Add ress	funct
100011	01001	01000	0000 0100 1011 0000		
000000	10010	01000	01000	00000	100000
101011	01001	01000	0000 0100 1011 0000		

Logical Instructions

Logical operations	C operators	Java operators	MIPS instructions
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR			or, ori
Bit-by-bit NOT	~	~	nor

- **sll \$t2, \$s0, 4** # reg \$t2 = reg \$s0 << 4 bits

Op	rs	rt	rd	shamt	func
0	0	16	10	4	0

- **AND** -> clear some bits whereas, **OR** -> to set some bits
- MIPS also provides the instructions *and immediate* (andi) and *or immediate* (ori).
- **NOR: A NOR 0 = NOT (A OR 0) = NOT (A).**

Decision-Making Instructions OR Conditional Jumps

- MIPS assembly language includes two decision-making instructions, traditionally called **conditional branches**.

beq register1, register2, L1

bne register1, register2, L1

- **Beq:** $\text{if}(\text{rs} == \text{rt}) \text{pc} += \text{offset} * 4$ (**PC-relative addressing**)
- **No. of instructions to be skipped** mentioned here as **offset** and not the actual bytes count. **Offset x 4** and then **add with PC**

Decision Making Instructions

- For greater or less comparisons,

slt \$t0, \$s3, \$s4 # \$t0 = 1 if \$s3 < \$s4

slti \$t0,\$s2,10 # \$t0 = 1 if \$s2 < 10

- MIPS compilers use the **slt**, **slti**, **beq**, **bne**, and the fixed value of 0 (always available by reading register **\$zero**) to create all relative conditions: equal, not equal, less than, less than or equal
- There **isn't a status register (Flag Register)**. Instead, the results of a comparison set a register value. The branch then tests this register value.

Decision Making Instructions

if (i == j) f = g + h; else f = g - h;

If f, g, h, i, and j are variables corresponding to the five registers \$s0 through \$s4

```
bne $s3,$s4,Else      # go to Else if i ≠ j
add $s0,$s1,$s2        # f = g + h (skipped if i ≠ j)
j Exit                 # go to Exit
Else: sub $s0,$s1,$s2  # f = g - h (skipped if i = j)
Exit:
```

Jump Instructions

Type	-31-	format (bits)					-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)	
I	opcode (6)	rs (5)	rt (5)	immediate (16)			
J	opcode (6)	address (26)					

- **J target**
 - $pc = pc_upper | (target \ll 2)$
- **JAL target**
 - Jump and link, used in procedure calls, save address of next instruction before jump.
 - $(\$ra)r31 = pc + 4; \quad pc = pc_upper | target \ll 2$
- **Jr \$rs (Not a J type Instr. Instead it's a R-type Instruction)**
 - Unconditional jump to address specified in register
 - $pc = rs$

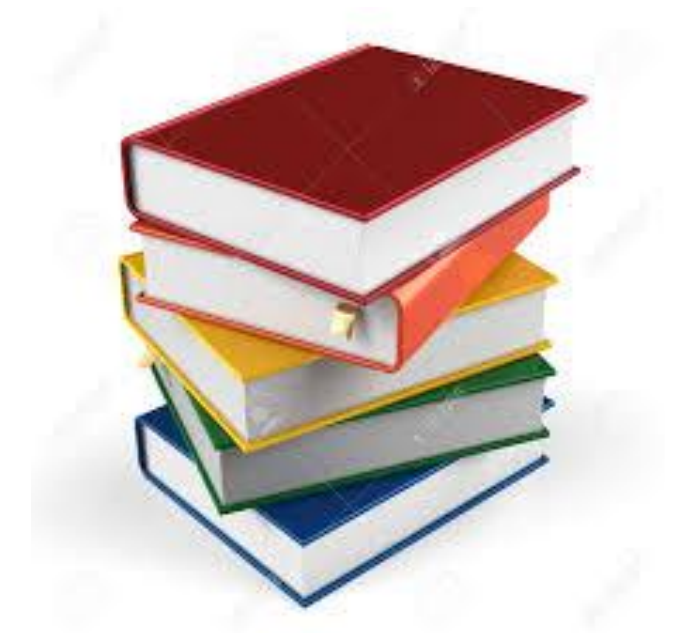
Op	rs	rt	rd	shamt	func
0	x	0	0	0	8

➤ PC reg. holds the address of the instr. that is going to be executed.

Supporting **Procedures** In Computer Hardware

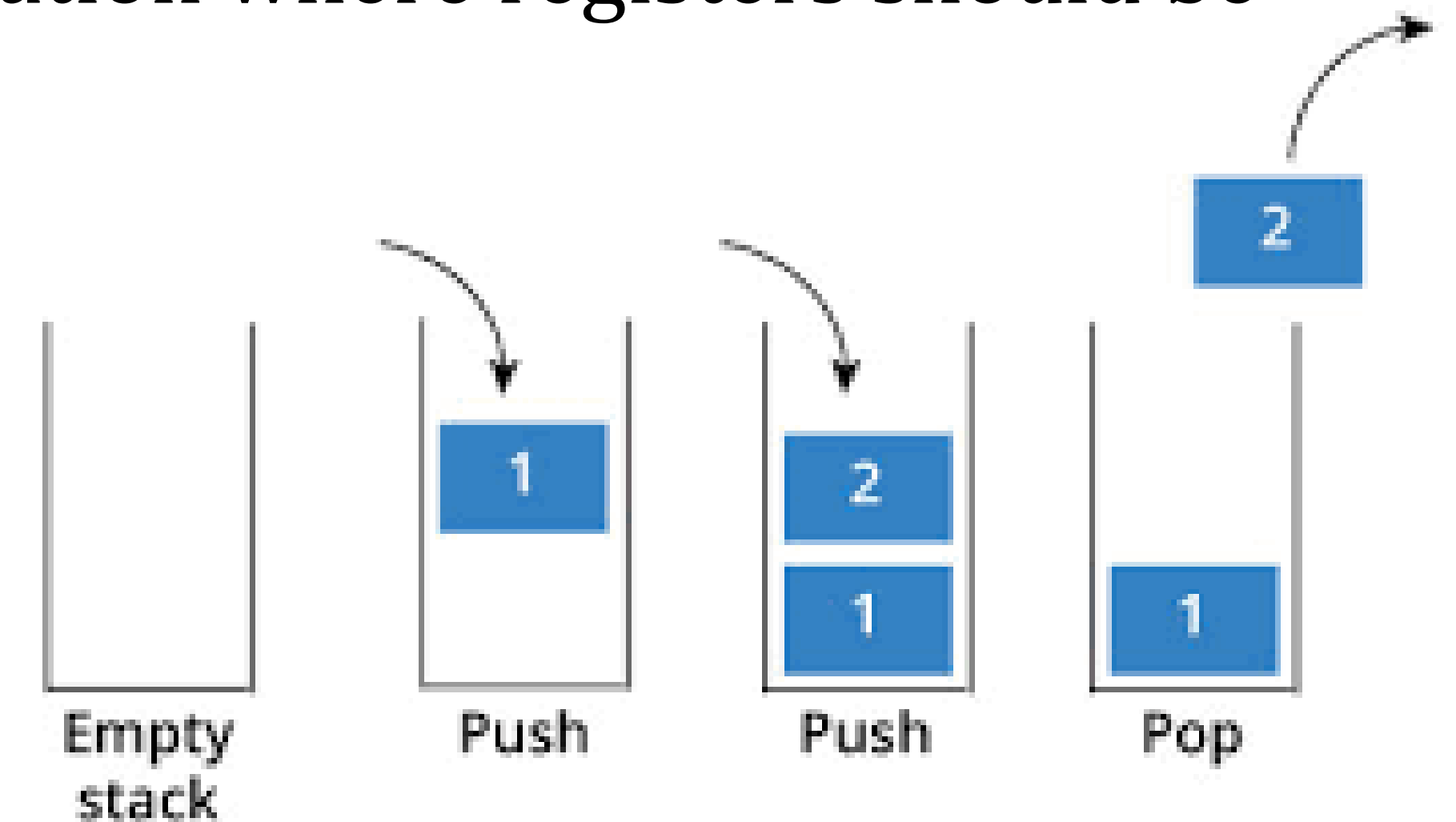
- MIPS used following conventions for Procedure Calling,
 - **\$a0 - \$a3**: four argument registers
 - **\$v0-\$v1**: two return value registers
 - **\$ra**: one return address register
- **Jal ProcedureAddress**: is used to jump to procedure and simultaneously saving the address of next instruction.
- **Jr \$ra**: Jump register instruction is used to return control back to the caller

Stacks



➤ If compiler needs more registers for a Procedure, then what?

- **Stack:** A data structure for spilling registers organized as a last-infirst-out queue.
- **Stack Pointer (\$SP, r29):** holds the address of stack location where registers should be spilled or where old register values can be found.
- **PUSH:** Add element to stack.
- **POP:** Remove element from the stack.



- Historically, stacks “grow” from higher addresses to lower addresses.

Procedures Example

Compiling a **Leaf_procedure** (that doesn't call other procedures)

```
int leaf_example (int g, int h, int i, int j)
{
    int f ;
    f = (g + h) - (i + j) ;
    return f ;
}
```

The parameter variables **g, h, i, and j** correspond to the argument registers **\$a0, \$a1, \$a2, and \$a3**, and **f corresponds to \$s0**. The compiled program starts with the label of the procedure:

Procedures Example

leaf_example:

```
addi $sp, $sp, -12    # adjust stack to make room for 3 items
sw $t1, 8($sp)         # save register $t1 for use afterwards
sw $t0, 4($sp)         # save register $t0 for use afterwards
sw $s0, 0($sp)         # save register $s0 for use afterwards

add $t0,$a0,$a1        # register $t0 contains g + h
add $t1,$a2,$a3        # register $t1 contains i + j
sub $s0, $t0, $t1      # f = $t0 - $t1, which is (g + h) - (i + j)
```


Procedures Example

returns f ($\$v0 = \$s0 + 0$) **MOVE instr.**

add $\$v0, \$s0, \$zero$

restore registers $\$s0, \$t0, \$t1$ for caller

lw $\$s0, 0(\$sp)$

lw $\$t0, 4(\$sp)$

lw $\$t1, 8(\$sp)$

adjust stack to delete 3 items

addi $\$sp, \$sp, 12$

jump back to calling routine

jr $\$ra$

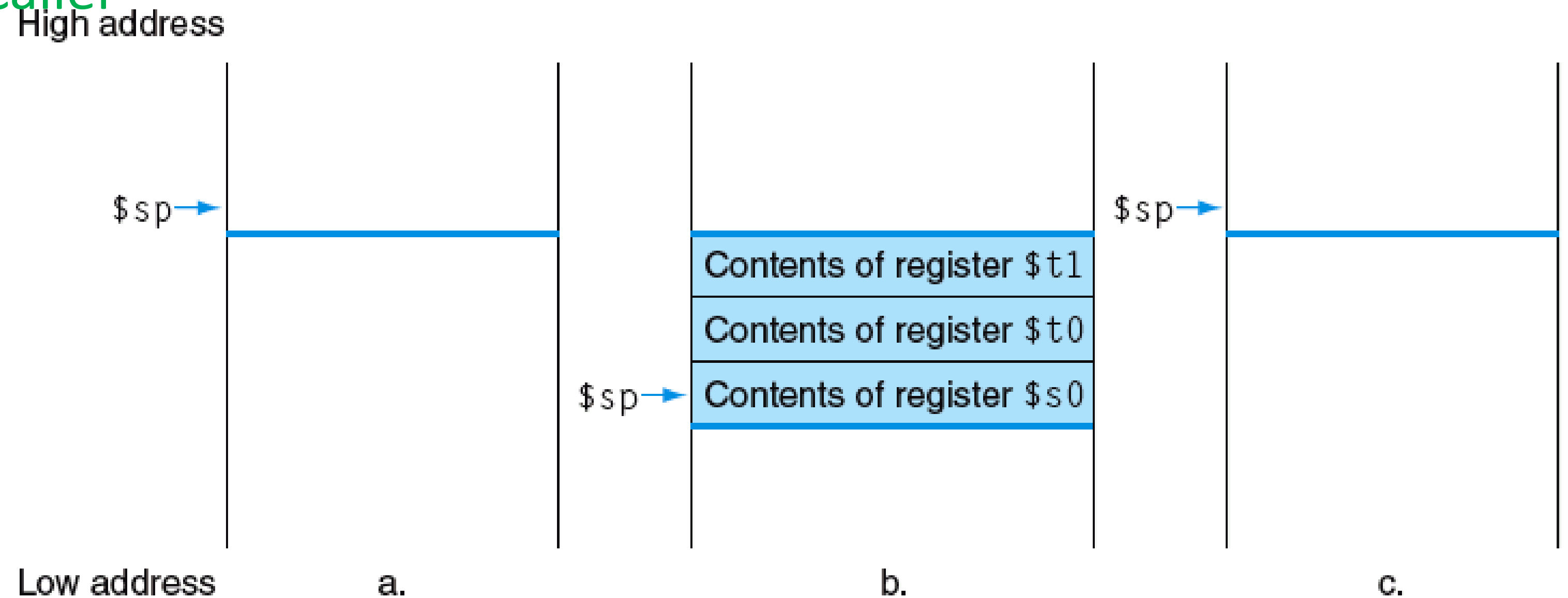


FIGURE 2.14 The values of the stack pointer and the stack (a) before, (b) during, and (c) after the procedure call. The stack pointer always points to the “top” of the stack, or the last word in the stack in this drawing.

MIPS Register Usage Conventions

Register	Name	Function	Reserved
R0	\$zero	Always contains constant 0	N.A.
R1	\$at	Reserved for Assembler temporary	N.A.
R2-R3	\$v0-\$v1	Expression evaluation and procedure return values	No
R4-R7	\$a0-\$a3	Function arguments	No
R8-R15	\$t0-\$t7	For temporary values	No
R16-R23	\$s0-\$s7	Saved registers across function calls	Yes
R24-R25	\$t8-\$t9	More temporary values	No
R26-R27	\$k0-\$k1	Reserved for interrupt handler / OS	N.A
R28	\$gp	Global pointer	Yes
R29	\$sp	Stack Pointer	Yes
R30	\$fp	Frame pointer	Yes
R31	\$ra	Return address from function call	Yes
HI-LO	lo-hi	Multiplication/division results	--
PC	Program Counter	Points at bytes past current instruction	--

Procedures Example

Compiling a **Nested_procedure** (Non-leaf procedure)

```
int fact (int n)
{
    if (n < 1)
        return (1);
    else
        return (n* fact(n - 1));
}
```

The parameter variables **n** correspond to the argument registers **\$a0**.

What is the MIPS assembly code?

Procedures Example

fact:

addi \$sp, \$sp, -8 # adjust stack to make room for 2 items

sw \$ra, 4(\$sp) # save the return address

sw \$a0, 0(\$sp) # save the argument n

Slti \$t0,\$a0,1 # test for $n < 1$

beq \$t0,\$zero, L1 # if $n \geq 1$, go to L1

addi \$v0, \$zero, 1 # return 1

Addi \$sp, \$sp, 8 # pop 2 item off stack

Jr \$ra # return to caller

Procedures Example

L1:

addi \$a0, \$a0, -1 # $n \geq 1$: argument gets (n-1)

Jal fact # call fact with (n-1)

the next instr. is where fact returns

lw \$a0, 0(\$sp) # return from jal: restore argument n

lw \$ra, 4(\$sp) # restore the return address

addi \$sp, \$sp, 8 #adjust stack pointer to pop 2 items

mul \$v0, \$a0, \$v0 # return $n * \text{fact}(n-1)$

Jr \$ ra # return to the caller

Character Representations

ASCII Codes (American Standard Code for Information Interchange)

- Most popular encoding scheme
- Only 7 bits are used for ASCII codes -> 128 ASCII Codes
- 0-31 and 127 are for communication control purposes
- 32-126 -> 95 ASCII codes are printable
- E.g. A = 41h , Z = 59h, a = 61h, Space = 20h etc.